

TREE 1

PROBLEM 1

QUESTION:

DS	Session	Tree 1	Question Information	Level 1	Challenge 61
Question description Jegan Sir students were chatting and playing quite loudly on the last day of the year, celebrating the end of the academic session. Jegan sir was harshly chastised by the college's principal. But, instead of becoming enraged, he attempted to engage everyone in a different task. So Jegan sir gave his students to solve the task such that, you have to perform pre-order tree traversal in Binary search tree.					

```
graph TD; 1((1)) --- 2((2)); 1 --- 3((3)); 2 --- 4((4)); 2 --- 5((5)); 3 --- 6((6)); 3 --- 7((7)); 4 --- 8((8)); 4 --- 9((9)); 6 --- 10((10)); 6 --- 11((11)); 7 --- 12((12));
```

Direction : Anti Clock

Rule : CLR (Center-Left-Right)

PreOrder:

1 2 4 8 9 5 3 6 10 7 11 12

Constraints:

0<size <100

0<data<1000

Input format:

First line indicates the size of the tree

Second line indicates the elements of the tree

Output Format:

single line indicates the pre-order list of the given value

Logical Test Cases Test Case 1 INPUT string 7 200 120 80 360 280 240 320 EXPECTED OUTPUT 200 120 80 360 280 240 320 Test Case 2 INPUT string 9 34 67 10 90 410 810 40 20 60 EXPECTED OUTPUT 34 10 20 67 40 60 90 40 410 810		
Mandatory Test Cases Test Case 1 KEYWORD void preorder(struct node* root) Test Case 2 KEYWORD struct node* newNode(int item) Test Case 3 KEYWORD struct node Test Case 4 KEYWORD struct node *left,*right;		
Complexity Test Cases Test Case 1 CHROMATIC COMPLEXITY 2 Test Case 2 TOKEN COUNT 200 Test Case 3 NLOC 50		

SOURCE CODE:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct BstNode {
    int key;
    struct BstNode *lChild;
    struct BstNode *rChild;
} BstNode;

BstNode* create_leaf(int val) {
    BstNode* node = (BstNode*)malloc(sizeof(BstNode));
    node->key = val;
    node->lChild = node->rChild = NULL;
    return node;
}

void insert_bst(BstNode** root_ref, int val) {
    if (*root_ref == NULL) {
        *root_ref = create_leaf(val);
        return;
    }
    if (val < (*root_ref)->key) {
        insert_bst(&(*root_ref)->lChild, val);
    } else if (val > (*root_ref)->key) {
        insert_bst(&(*root_ref)->rChild, val);
    }
}

void print_preorder(const BstNode* root) {
    if (!root) return;
    printf("%d ", root->key);
    print_preorder(root->lChild);
    print_preorder(root->rChild);
}

int main() {
    int count;
    if (scanf("%d", &count) != 1) return 0;

    BstNode* root = NULL;
    for (int i = 0; i < count; i++) {
        int element;
        scanf("%d", &element);
        insert_bst(&root, element);
    }
    print_preorder(root);
    printf("\n");
    return 0;
}
```

OUTPUT:

```
9
34 67 10 90 410 810 40 20 60
34 10 20 67 40 60 90 410 810
```

```
7
200 120 80 160 280 240 320
200 120 80 160 280 240 320
```